

# PPO-CMA: PROXIMAL POLICY OPTIMIZATION WITH COVARIANCE MATRIX ADAPTATION

Perttu Hämmäläinen\*    Amin Babadi\*    Xiaoxiao Ma\*    Jaakko Lehtinen\*†

\* Aalto University, Helsinki, Finland

† NVIDIA

## ABSTRACT

Proximal Policy Optimization (PPO) is a highly popular model-free reinforcement learning (RL) approach. However, we observe that in a continuous action space, PPO can prematurely shrink the exploration variance, which leads to slow progress and may make the algorithm prone to getting stuck in local optima. Drawing inspiration from CMA-ES, a black-box evolutionary optimization method designed for robustness in similar situations, we propose PPO-CMA, a proximal policy optimization approach that adaptively expands the exploration variance to speed up progress. With only minor changes to PPO, our algorithm considerably improves performance in Roboschool continuous control benchmarks. Our results also show that PPO-CMA, as opposed to PPO, is significantly less sensitive to the choice of hyperparameters, allowing one to use it in complex movement optimization tasks without requiring tedious tuning.

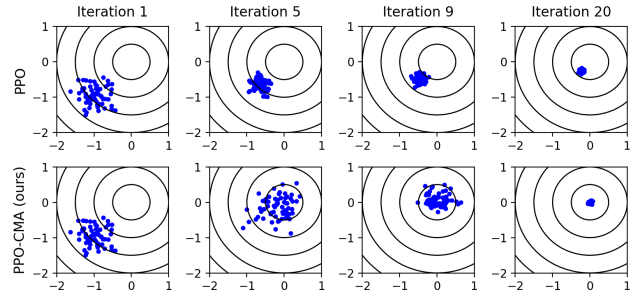
**Index Terms**— Continuous Control, Reinforcement Learning, Policy Optimization, Policy Gradient, Evolution Strategies, CMA-ES, PPO

## 1. INTRODUCTION

Policy optimization with continuous state and action spaces is a central, long-standing problem in robotics and computer animation. In the general case, one does not have a differentiable model of the dynamics and must proceed by trial and error, i.e., try something (sample actions from an exploration distribution, e.g., a neural network policy conditioned on the current state), see what happens, and learn from the results (update the exploration distribution such that good actions become more probable). In recent years, such approaches have achieved remarkable success in previously intractable tasks such as real-time locomotion control of (simplified) biomechanical models of the human body [1]. One of the most popular policy optimization algorithms to achieve this is Proximal Policy Optimization (PPO) [1, 2, 3].

In this paper, we make the following contributions:

- We provide novel evidence of how PPO’s exploration variance can shrink prematurely, which leads to slow progress. Figure 1 illustrates this in a simple didactic problem.
- We propose PPO-CMA, a method that dynamically expands and contracts the exploration variance, inspired by the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) optimization method. This only requires minor changes to vanilla PPO but improves performance considerably.



**Fig. 1.** Comparing PPO and PPO-CMA with a simple “stateless” quadratic objective. Sampled actions  $\mathbf{a} \in \mathbb{R}^2$  are shown in blue. PPO shrinks the sampling/exploration variance prematurely, which leads to slow final progress. The proposed PPO-CMA method dynamically expands the variance to speed up progress, and only shrinks the variance when close to the optimum. Source code and animated visualization can be found at: <https://github.com/ppocma/ppocma>.

## 2. RELATED WORK

Our work is closely related to Continuous Actor Critic Learning Automaton (CACLA) [4]. CACLA uses the sign of the advantage estimate – in their case the TD-residual – in the updates, shifting policy mean towards actions with positive sign. The paper also observes that using actions with negative advantages can have an adverse effect. However, using only positive advantage actions guarantees that the policy stays in the proximity of the collected experience. Thus, CACLA can be viewed as an early PPO approach, which we extend with CMA-ES -style variance adaptation.

PPO represents on-policy RL methods, i.e., experience is assumed to be collected on-policy and thus must be discarded after the policy is updated. Theoretically, off-policy RL should allow better sample efficiency through the reuse of old experience, often implemented using an experience replay buffer. PPO-CMA can be considered as a hybrid method, since the policy mean is updated using on-policy experience, but the history or replay buffer for the variance update also includes older off-policy experience.

CMA-ES has been also applied to continuous control in the form of trajectory optimization. In this case, one can use CMA-ES to search for a sequence of optimal controls given an initial state [5]. Although trajectory optimization approaches have demonstrated impressive results with complex humanoid characters, they require more computing resources in run-time. Trajectory optimization has

also been leveraged to inform policy search using the principle of maximum entropy control [6], which leads to a Gaussian policy. Furthermore, Differential Dynamic Programming (DDP) has been formulated in the terms of Gaussian distributions, which permits using CMA-ES for sampling the actions of each timestep in trajectory optimization [7].

PPO-CMA is perhaps most closely related to [8, 9]. Maximum a posteriori Policy Optimization (MPO) [8] also fits the policy to the collected experience using a weighted maximum likelihood approach, but negative weights are avoided through exponentiated Q-values, based on the control-inference dualism, instead of our negative advantage mirroring. Concurrently with our work, MPO has also been extended with decoupled optimization of policy mean and variance, yielding similar variance adaptation behaviour as CMA-ES and PPO-CMA; on a quadratic objective, variance is first increased and shrinks only when close to the optimum [9].

Finally, it should be noted that PPO-CMA falls in the domain of model-free reinforcement learning approaches. In contrast, there are several model-based methods that learn approximate models of the simulation dynamics and use the models for policy optimization, potentially requiring less simulated or real experience. Both ES and RL approaches can be used for the optimization. Model-based algorithms are an active area of research, with recent work demonstrating excellent results in limited MuJoCo benchmarks [10], but model-free approaches still dominate the most complex continuous problems such as humanoid movement.

### 3. PRELIMINARIES

#### 3.1. Reinforcement Learning

We consider the discounted formulation of the policy optimization problem, following the notation of [11]. At time  $t$ , the agent observes a state vector  $\mathbf{s}_t$  and takes an action  $\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ , where  $\pi_\theta$  denotes the policy parameterized by  $\theta$ , e.g., neural network weights. We focus on on-policy methods where the optimized policy also defines the exploration distribution. Executing the sampled action results in observing a new state  $\mathbf{s}'_t$  and receiving a scalar reward  $r_t$ . The goal is to find  $\theta$  that maximizes the expected future-discounted sum of rewards  $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$ , where  $\gamma$  is a discount factor in the range  $[0, 1]$ . A lower  $\gamma$  makes the learning prefer instant gratification instead of long-term gains.

Both PPO and the PPO-CMA collect experience tuples  $[\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i]$  by simulating a number of *episodes* in each optimization iteration. For each episode, an initial state  $\mathbf{s}_0$  is sampled from some application-dependent stationary distribution, and the simulation is continued until a terminal (absorbing) state or a predefined maximum episode length  $T$  is reached. After the iteration simulation budget  $N$  is exhausted,  $\theta$  is updated.

#### 3.2. Policy Gradient with Advantage Estimation

Policy gradient methods update policy parameters by estimating the gradient  $\mathbf{g} = \nabla_\theta \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$ . PPO utilizes the following policy gradient loss:

$$\mathcal{L}_\theta = -\frac{1}{M} \sum_{i=1}^M A^\pi(\mathbf{s}_i, \mathbf{a}_i) \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i), \quad (1)$$

where  $i$  denotes minibatch sample index and  $M$  is minibatch size.  $A^\pi(\mathbf{s}_i, \mathbf{a}_i)$  denotes the *advantage function*, which measures the benefit of taking action  $\mathbf{a}_i$  in state  $\mathbf{s}_i$ . Positive  $A^\pi$  means that the action was better than average and minimizing the loss function will

increase the probability of sampling the same action again. Note that  $A^\pi$  does not directly depend on  $\theta$  and thus acts as a constant when computing the gradient of Equation 1. A maximum likelihood view of Equation 1 is that the policy distribution is fitted to the data, each data point weighted by its advantage [12]. Same as PPO, we use Generalized Advantage Estimation (GAE) [11], a simple but effective way to estimate  $A^\pi$ .

#### 3.3. Continuous Action Spaces

With a continuous action space, it is common to use a Gaussian policy. In other words, the policy network outputs state-dependent mean  $\mu_\theta(\mathbf{s})$  and covariance  $\mathbf{C}_\theta(\mathbf{s})$  for sampling the actions. The covariance defines the exploration-exploitation balance. In the most simple case of isotropic unit Gaussian exploration,  $\mathbf{C} = \mathbf{I}$ , the loss function in Equation 1 becomes:

$$\mathcal{L}_\theta = \frac{1}{M} \sum_{i=1}^M A^\pi(\mathbf{s}_i, \mathbf{a}_i) \|\mathbf{a}_i - \mu_\theta(\mathbf{s}_i)\|^2, \quad (2)$$

Intuitively, minimizing the loss drives the policy mean towards positive-advantage actions and away from negative-advantage actions.

Following the original PPO paper, we use a diagonal covariance matrix parameterized by a vector  $\mathbf{c}_\theta(\mathbf{s}) = \text{diag}(\mathbf{C}_\theta(\mathbf{s}))$ . In this case, the loss becomes:

$$\begin{aligned} \mathcal{L}_\theta = & \frac{1}{M} \sum_{i=1}^M A^\pi(\mathbf{s}_i, \mathbf{a}_i) \sum_j \left[ \frac{(a_{i,j} - \mu_{j;\theta}(\mathbf{s}_i))^2}{c_{j;\theta}(\mathbf{s}_i)} \right. \\ & \left. + 0.5 \log c_{j;\theta}(\mathbf{s}_i) \right], \end{aligned} \quad (3)$$

where  $i$  indexes over a minibatch and  $j$  indexes over action variables.

#### 3.4. Proximal Policy Optimization

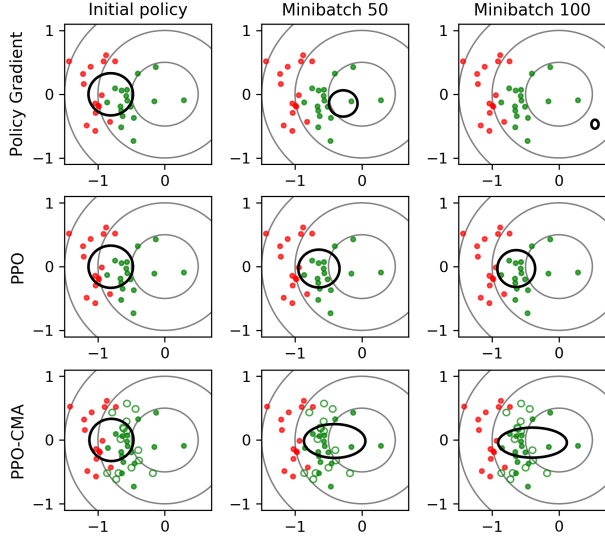
The basic idea of PPO is that one performs not just one but multiple minibatch gradient steps with the experience of each iteration. Essentially, one reuses the same data to make more progress per iteration, while stability is ensured by limiting the divergence between the old and updated policies [3]. PPO is a simplification of Trust Region Policy Optimization (TRPO) [13], which uses a more computationally expensive approach to achieve the same.

The original PPO paper [3] proposes two variants: 1) using an additional loss term that penalizes KL-divergence between the old and updated policies, and 2) using the so-called clipped surrogate loss function. The paper concludes that the clipped surrogate loss is the recommended choice. This is also the version that we use in this paper in all PPO vs. PPO-CMA comparisons.

### 4. PROBLEM ANALYSIS AND VISUALIZATION

To allow simple visualization of actions sampled from the policy, the didactic problem in Figure 1 simplifies policy optimization into a generic black-box optimization problem in a 2D action space.

To achieve this, we set  $\gamma = 0$ , which simplifies the policy optimization objective  $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t] = \mathbb{E}[r_0] = \mathbb{E}[r(\mathbf{s}, \mathbf{a})]$ , where  $\mathbf{s}$  and  $\mathbf{a}$  denote the first state and action of an episode. Thus, we can use  $T = 1$  and focus on visualizing only the first timesteps of each episode. Further, we use a state-agnostic  $r(\mathbf{s}, \mathbf{a}) = r(\mathbf{a}) = -\mathbf{a}^T \mathbf{a}$ . Thus, we have a simple quadratic optimization problem and it is enough to only visualize the action space. The optimal policy Gaussian has zero mean and variance.



**Fig. 2.** Policy evolution over the minibatch gradient steps of a single iteration in our didactic problem. Figure 1 shows how the same methods perform over multiple iterations. The black ellipses denote the policy mean and standard deviation according to which actions are sampled/explored. Positive-advantage actions are shown in green, negative advantages in red. The green non-filled circles show the negative-advantage actions converted to positive ones (Section 5.2). Policy gradient diverges. PPO limits the update before convergence or divergence. PPO-CMA expands the variance in the progress direction, improving exploration in subsequent iterations.

#### 4.1. The Instability Caused by Negative Advantages

Considering the Gaussian policy gradient loss functions in Equations 2 and 3, we note a fundamental problem: *Actions with a negative advantage may cause instability* when performing multiple minibatch gradient steps in PPO style, as each step drives the policy Gaussian further away from the negative-advantage actions. This visualized in Figure 2 (top row). The policy diverges, gravitating away from the negative-advantage actions.

Note that policy gradient does not diverge if one collects new experience and re-estimates the advantages between each gradient step; this, however, is what PPO aims to avoid for better sample efficiency.

#### 4.2. PPO: Stable But May Converge Prematurely

As visualized in Figure 1, the clipped surrogate loss of PPO prevents divergence, but the exploration variance can shrink prematurely when performing the updates over multiple iterations. Possibly to mitigate this, PPO also adds an entropy loss term that penalizes low variance. However, the weight of the entropy loss can be difficult to finetune.

Although the original PPO paper [3] demonstrated good results in MuJoCo problems with a Gaussian policy, the most impressive Roboschool results did not adapt the variance through gradient updates. Instead, the policy network only output the Gaussian mean and a linearly decaying variance with manually tuned decay rate was used. Thus, our observation of variance adaptation issues complements their work instead of contradicting it.

#### 4.3. How CMA-ES Solves the Problems

The Evolution Strategies (ES) literature has a long history of solving similar Gaussian exploration and variance adaptation problems, culminating in the widely used CMA-ES optimization method and its recent variants [14, 15]. CMA-ES is a black-box optimization method for finding a parameter vector  $\mathbf{x}$  that maximizes some objective or fitness function  $f(\mathbf{x})$ . The key difference to PPO is that the exploration distribution's mean and covariance are plain variables instead of neural networks, and not conditioned on an agent's current state. However, as we propose in Section 5, one can adapt the key ideas of CMA-ES to policy optimization.

The CMA-ES core iteration is summarized in Algorithm 1. Although there is no convergence guarantee, CMA-ES performs remarkably well on multimodal and/or noisy functions if using enough samples per iteration. For full details of the update rules, the reader is referred to Hansen's excellent tutorial [16].

---

##### Algorithm 1 High-level summary of CMA-ES

---

- 1: **for** iteration=1,2,... **do**
  - 2:   Draw samples  $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{C})$ .
  - 3:   Evaluate  $f(\mathbf{x}_i)$ .
  - 4:   Sort the samples based on  $f(\mathbf{x}_i)$  and compute weights  $\mathbf{w}_i$  based on the ranks, such that best samples have highest weights.
  - 5:   Update  $\boldsymbol{\mu}$  and  $\mathbf{C}$  using the samples and weights.
  - 6: **end for**
- 

CMA-ES avoids premature convergence and instability through the following:

1. When fitting the sampling distribution to weighted samples, only positive weights are used.
2. The so-called rank- $\mu$  update and evolution path heuristic elongate the exploration variance in the progress direction, making premature convergence less likely.

Below, we overview these key ideas, providing a foundation for Section 5, which adapts the ideas for episodic RL. Note that CMA-ES is not directly applicable to RL, as instead of a single action optimization task, RL is in effect solving multiple action optimization tasks in parallel, one for each possible state. With a continuous state space, one cannot enumerate the samples for each state, which means that the sorting operation of Algorithm 1 is not feasible.

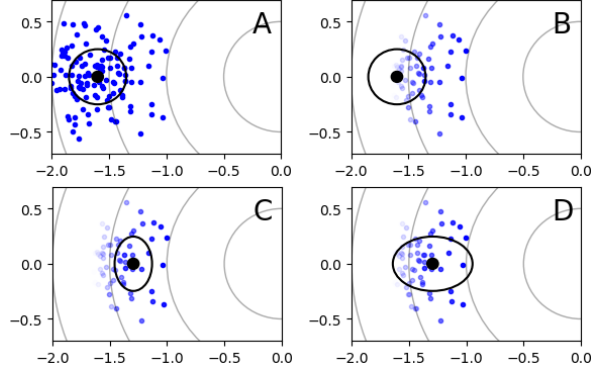
##### 4.3.1. Computing Sample Weights

Using the default CMA-ES parameters, the weights of the worst 50% of samples are set to 0, i.e., samples below median fitness are pruned and have no effect. The exploration mean  $\boldsymbol{\mu}$  is updated in maximum likelihood manner to weighted average of the samples (similar to minimizing the loss in Equation 2 with the advantages as the weights). Because CMA-ES uses only non-negative weights, the maximum likelihood update does not diverge from the sampled actions. This suggests that one could similarly only use positive advantage actions (the better half of the explored actions) in PPO.

##### 4.3.2. The rank- $\mu$ update

Superficially, the core iteration loop of CMA-ES is similar to other optimization approaches with recursive sampling and distribution fitting such as the Cross-Entropy Method [17] and Estimation of Multivariate Normal Algorithm (EMNA) [18]. However, there is

a crucial difference: in the so-called Rank- $\mu$  update, *CMA-ES first updates the covariance and only then updates the mean* [16]. This has the effect of elongating the exploration distribution along the best search directions instead of shrinking the variance prematurely, as shown in Figure 3.



**Fig. 3.** The difference between joint and separate updating of mean and covariance, denoted by the black dot and ellipse. A) sampling, B) pruning and weighting of samples based on fitness, C) EMNA-style update, i.e., estimating mean and covariance based on weighted samples, D) CMA-ES rank- $\mu$  update, where covariance is estimated before updating the mean. This elongates the variance in the progress direction, improving exploration in the next iteration.

#### 4.3.3. Evolution path heuristic

CMA-ES also features the so-called evolution path heuristic, where a component  $\alpha \mathbf{p}^{(i)} \mathbf{p}^{(i)T}$  is added to the covariance, where  $\alpha$  is a scalar, the  $(i)$  superscript denotes iteration index, and  $\mathbf{p}$  is the evolution path [16]:

$$\mathbf{p}^{(i)} = \beta_0 \mathbf{p}^{(i-1)} + \beta_1 (\boldsymbol{\mu}^{(i)} - \boldsymbol{\mu}^{(i-1)}). \quad (4)$$

Although the exact computation of the default  $\beta_0$  and  $\beta_1$  multipliers is rather involved, Equation 4 essentially amounts to first-order low-pass filtering of the steps taken by the distribution mean  $\boldsymbol{\mu}$  between iterations. When CMA-ES progresses along a continuous slope of the fitness landscape,  $\|\mathbf{p}\|$  is large, and the covariance is elongated and exploration is increased along the progress direction. Near convergence, when CMA-ES zigzags around the optimum in a random walk,  $\|\mathbf{p}\| \approx 0$  and the evolution path heuristic has no effect.

## 5. PPO-CMA

Building on the previous section, we can now describe our PPO-CMA method, summarized in Algorithm 2. PPO-CMA is simple to implement, only requiring the following changes to PPO:

- Instead of the clipped surrogate loss, we use the standard policy gradient loss in Equation 3 and train only on actions with positive advantage estimates to ensure stability, as motivated in Section 4.3.1. However, as setting negative advantages to zero discards information, we also propose a mirroring technique for converting negative-advantage actions to positive ones (Section 5.2).

- We implement the rank- $\mu$  update (Section 4.3.2) using separate neural networks for policy mean and variance. This way, the variance can be updated before updating the mean.
- We maintain a history of training data over  $H$  iterations, used for training the variance network. This approximates the CMA-ES evolution path heuristic, as explained below.

Together, these features result in the emergence of PPO-CMA variance adaptation behavior shown in Figures 1 and 2. Despite the differences to original PPO, we still consider PPO-CMA a proximal policy optimization method, as fitting the new policy to the positive-advantage actions prevents it diverging far from the current policy.

### 5.1. Approximating the Evolution Path Heuristic

We approximate the CMA-ES evolution path heuristic (Section 4.3.3) by keeping a history of  $H$  iterations of data and sampling the variance training minibatches from the history instead of only the latest data. Similar to the original evolution path heuristic, this elongates the variance for a given state if the mean is moving in a consistent direction. We do not implement the CMA-ES evolution path heuristic directly, because this would need yet another neural network to maintain and approximate a state-dependent  $\mathbf{p}(\mathbf{s})$ . Similar to exploration mean and variance,  $\mathbf{p}$  is a CMA-ES algorithm state variable; in policy optimization, such variables become functions of agent state and need to be encoded as neural network weights.

### 5.2. Mirroring Negative-Advantage Actions

Disregarding negative-advantage actions may potentially discard valuable information. We observe that assuming linearity of advantage around the current policy mean  $\boldsymbol{\mu}(\mathbf{s}_i)$ , it is possible to mirror negative-advantage actions about the mean to convert them to positive-advantage actions. More precisely, we set  $\mathbf{a}'_i = 2\boldsymbol{\mu}(\mathbf{s}_i) - \mathbf{a}_i$ ,  $A^\pi(\mathbf{a}'_i) = -A^\pi(\mathbf{a}_i)\psi(\mathbf{a}_i, \mathbf{s}_i)$ , where  $\psi(\mathbf{a}_i, \mathbf{s}_i)$  is a Gaussian kernel (we use the same shape as the policy) that assigns less weight to actions far from the mean. This is visualized at the bottom of Figure 2. The mirroring drives the policy Gaussian away from worse than average actions, but in a way consistent with the weighted maximum likelihood estimation perspective which requires non-negative weights for stability. If the linearity assumption holds, the mirroring effectively doubles the amount of data informing the updates.

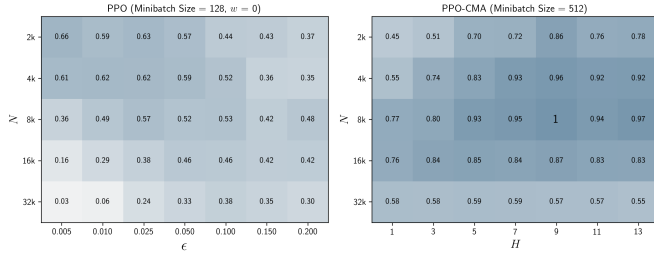
---

#### Algorithm 2 PPO-CMA

---

- 1: **for** iteration=1,2,... **do**
  - 2:   **while** iteration simulation budget  $N$  not exceeded **do**
  - 3:     Reset the simulation to a (random) initial state
  - 4:     Run agent on policy  $\pi_\theta$  for  $T$  timesteps or until a terminal state
  - 5:   **end while**
  - 6:   Train critic network for  $K$  minibatches using the experience from the current iteration
  - 7:   Estimate advantages  $A^\pi$  using GAE [11]
  - 8:   Clip negative advantages to zero,  $A^\pi \leftarrow \max(A^\pi, 0)$  or convert them to positive ones (Section 5.2)
  - 9:   Train policy variance for  $K$  minibatches using experience from past  $H$  iterations and Eq. 3
  - 10:   Train policy mean for  $K$  minibatches using the experience from this iteration and Eq. 3
  - 11: **end for**
-





**Fig. 4.** PPO and PPO-CMA performance as a function of key hyperparameters, using average normalized scores from 9 Roboschool environments (1 is the best observed score). The batch sizes and entropy loss weight  $w$  values are the ones that produced the best results in our hyperparameter searches. PPO-CMA performs overall better, is not as sensitive to the hyperparameter choices, and the hyperparameters can be adjusted more independently. In contrast, PPO requires careful finetuning of both the  $\epsilon$  and  $N$  parameters.

In the CMA-ES literature, a related technique is to use a negative covariance matrix update procedure [19], but the technique does not improve the estimation of the mean.

## 6. EVALUATION

A key issue in the usability of an RL method is sensitivity to hyperparameters. As learning complex tasks can take hours or days, finetuning hyperparameters is tedious. Thus, we conducted hyperparameter searches to investigate following questions:

- Can PPO-CMA produce better results than PPO without precise tuning of hyperparameters?
- Can hyperparameters optimized for simple tasks generalize to complex tasks?
- Is PPO-CMA less prone to getting stuck in local optima?

As elaborated below, our data indicates a positive answer to all the questions. Furthermore, we conducted an ablation study that verifies that all the proposed PPO-CMA algorithm features improve the results.

We used the 9 2D OpenAI Gym Roboschool continuous control environments [20] for the hyperparameter searches and the ablation study and tested generalization on the OpenAI Gym MuJoCo Humanoid [21]. We performed 5 training runs with different random seeds for environment and setting (total 31500 runs). The same codebase was used for both PPO and PPO-CMA, only toggling PPO-CMA features on/off, to ensure no differences due to code-level optimizations [22].

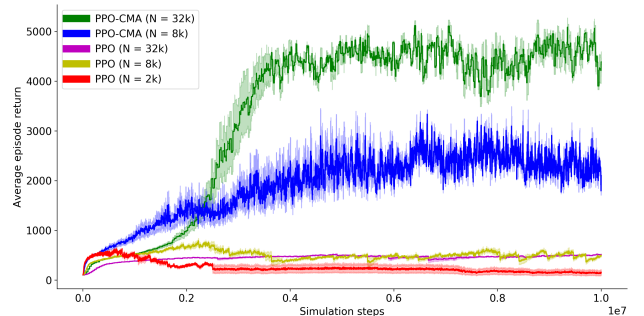
### 6.1. Sensitivity to Hyperparameters

Figure 4 visualizes sensitivity to key hyperparameters, i.e., iteration simulation budget  $N$ , PPO-CMA’s history buffer size  $H$ , and PPO’s clipping parameter  $\epsilon$  that determines how much the updated policy can diverge from the old one. The scores in Figure 4 are normalized averages over all runs for a hyperparameter and algorithm choice, such that the best best parameter combination and algorithm has score 1. We tested minibatch sizes 128, 256, 512, 1024, 2048 and use the best performing ones for Figure 4. The figure reveals the following:

- PPO-CMA performs better with a wide range of hyperparameters, in particular with  $H \geq 5$ . Similar to CMA-ES, the main parameter to adjust is  $N$ . A large  $N$  makes progress more robust and less noisy. On the other hand, a large  $N$  means less iterations and possibly less progress within some total simulation budget, which shows as the lower scores for the largest  $N$  in Figure 4.
- There is a strong interaction of PPO’s  $\epsilon$  and  $N$ ; if one is changed, the other must be also changed. This makes finetuning the parameters difficult, especially considering that PPO has the additional entropy loss weight parameter to tune. Interestingly, the optimal parameter combination appears to be a very low  $\epsilon$  together with a very low  $N$ . On the other hand,  $N$  should not be decreased below the episode time limit  $T$ . Most of the Roboschool environments use  $T = 1000$ .

### 6.2. Generalization and Scaling to Complex Problems

The 9 environments used for the hyperparameter search are all relatively simple 2D tasks. To test generalization and scaling, we ran both algorithms on the more challenging MuJoCo Humanoid-v2 environment [21]. We used the best-performing hyperparameters of Figure 4 and also tested a larger simulation budget  $N$ . The agent is a 3D humanoid that gets rewards for forward locomotion. The results are shown in Figure 5. PPO-CMA yields clearly better results, especially with the increased  $N$ .



**Fig. 5.** Comparing PPO and PPO-CMA in the MuJoCo Humanoid-v2 environment, showing means and standard deviations of training curves from 3 runs with different random seeds.

### 6.3. Ablation study

To ensure that PPO-CMA has no redundant features, we tested different ablated versions on the 9 Roboschool environments used for Figure 4. Table 1 shows the resulting normalized scores. The results indicate that all the proposed algorithm components improve performance.

## 7. LIMITATIONS

We only compare PPO-CMA to PPO, and recognize that there are more recent RL algorithms that may offer better performance [8, 9, 10]. However, PPO is a widely used algorithm due to its combination of simplicity and good performance, which calls for better understanding of its limitations.

| Algorithm version                               | Score |
|-------------------------------------------------|-------|
| No mirroring, no evolution path, no rank- $\mu$ | 0.57  |
| No mirroring, no evolution path heuristic       | 0.71  |
| No mirroring of negative-advantage actions      | 0.82  |
| Full PPO-CMA                                    | 1     |

**Table 1.** Ablation study results, showing normalized scores similar to Figure 4. Note that not using the rank- $\mu$  heuristic amounts to using a single policy network that outputs both mean and variance.

## 8. CONCLUSION

Proximal Policy Optimization (PPO) is a simple, powerful, and widely used model-free reinforcement learning approach. However, we have observed that PPO can prematurely shrink the exploration variance, leading to slow convergence. As a solution to the variance adaptation problem, we have proposed the PPO-CMA algorithm that adopts the rank- $\mu$  update and evolution path heuristics of CMA-ES to episodic RL.

PPO-CMA improves PPO results in the tested Roboschool continuous control tasks while not sacrificing mathematical and conceptual simplicity. We add the separate neural networks for policy mean and variance and the  $H$  hyperparameter, but on the other hand, we do not need PPO’s clipped surrogate loss function, the  $\epsilon$  parameter, or the entropy loss term. Similar to CMA-ES, PPO-CMA can be said to be quasi-parameter-free; according to our experience, once the neural network architecture is decided on, one mainly needs to increase the iteration sampling budget  $N$  for more difficult problems.

## Acknowledgments

This work has been supported by Academy of Finland grants 299358 and 305737.

## 9. REFERENCES

- [1] Kevin Bergamin, Simon Clavet, Daniel Holden, and James Richard Forbes, “Drecon: data-driven responsive control of physics-based characters,” *ACM Transactions on Graphics (TOG)*, vol. 38, no. 6, pp. 1–11, 2019.
- [2] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne, “Deepmimic: Example-guided deep reinforcement learning of physics-based character skills,” *arXiv preprint arXiv:1804.02717*, 2018.
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [4] Hado van Hasselt and Marco A Wiering, “Reinforcement learning in continuous action spaces,” in *Approximate Dynamic Programming and Reinforcement Learning, 2007. ADPRL 2007. IEEE International Symposium on*. IEEE, 2007, pp. 272–279.
- [5] Amin Babadi, Kourosh Naderi, and Perttu Härmäläinen, “Intelligent middle-level game control,” in *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2018, pp. 1–8.
- [6] Sergey Levine and Vladlen Koltun, “Guided policy search,” in *International Conference on Machine Learning*, 2013, pp. 1–9.
- [7] Joose Rajamäki and Perttu Härmäläinen, “Regularizing sampled differential dynamic programming,” in *2018 Annual American Control Conference (ACC)*. IEEE, 2018, pp. 2182–2189.
- [8] Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller, “Maximum a posteriori policy optimisation,” *arXiv preprint arXiv:1806.06920*, 2018.
- [9] Abbas Abdolmaleki, Jost Tobias Springenberg, Jonas Degraeve, Steven Bohez, Yuval Tassa, Dan Belov, Nicolas Heess, and Martin Riedmiller, “Relative entropy regularized policy iteration,” *arXiv preprint arXiv:1812.02256*, 2018.
- [10] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine, “Deep reinforcement learning in a handful of trials using probabilistic dynamics models,” *arXiv preprint arXiv:1805.12114*, 2018.
- [11] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [12] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *CoRR*, vol. abs/1910.00177, 2019.
- [13] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz, “Trust region policy optimization,” in *International Conference on Machine Learning*, 2015, pp. 1889–1897.
- [14] Nikolaus Hansen, “The cma evolution strategy: a comparing review,” in *Towards a new evolutionary computation*, pp. 75–102. Springer, 2006.
- [15] Ilya Loshchilov, Tobias Glasmachers, and Hans-Georg Beyer, “Limited-memory matrix adaptation for large scale black-box optimization,” *arXiv preprint arXiv:1705.06693*, 2017.
- [16] Nikolaus Hansen, “The cma evolution strategy: A tutorial,” *arXiv preprint arXiv:1604.00772*, 2016.
- [17] Pieter-Tjerk De Boer, Dirk P Kroese, Shie Mannor, and Reuven Y Rubinstein, “A tutorial on the cross-entropy method,” *Annals of operations research*, vol. 134, no. 1, pp. 19–67, 2005.
- [18] Pedro Larrañaga and Jose A Lozano, *Estimation of distribution algorithms: A new tool for evolutionary computation*, vol. 2, Springer Science & Business Media, 2001.
- [19] Nikolaus Hansen and Raymond Ros, “Benchmarking a weighted negative covariance matrix update on the bbob-2010 noiseless testbed,” in *Proceedings of the 12th annual conference companion on Genetic and evolutionary computation*. ACM, 2010, pp. 1673–1680.
- [20] OpenAI, “Roboschool,” <https://blog.openai.com/roboschool/>, 2017.
- [21] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba, “Openai gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [22] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry, “Implementation matters in deep rl: A case study on ppo and trpo,” in *International Conference on Learning Representations*, 2019.